

Doc. No.: WG21/P0667R0

Date: 2017-6-18

Reply-to: Hans-J. Boehm

Email: hboehm@google.com

Audience: SG1, LEWG

P0667R0: The future of `std::future` extensions

The purpose of this brief paper is to facilitate discussion of whether, and in what form the futures extensions (Clause 2 [futures]) of the Concurrency TS should be moved into C++20. Many users feel that they are sorely needed. But, unlike the other parts of the TS, there appear to be deep concerns about including them unchanged. Thus this is intended to start discussion about the right path here.

Some concerns

I will outline my concerns. There may be others.

The central component of this section is the `.then()` member function of `std::future`. This runs the argument continuation when the future becomes ready. An earlier design took an explicit argument to specify where the continuation should be run. With the demise of the original executor proposal, this became impossible.

As is pointed out in LWG2533, the current TS is unclear where continuations are run. In the absence of executors, it seems reasonable to run continuations on whatever thread or, more generally, execution agent fulfilled the promise. But this is not possible if the future is ready at the point at which `.then()` is called. Thus we focus on this important, though somewhat unusual, case.

There seem to be at least two somewhat reasonable, but very different options. If `.then()` is used to string together a chain of I/O operations, well-designed client code will look very different in the two cases.

1. Since the execution agent satisfying the original promise is no longer available, we simply run the `.then()` continuation synchronously in the `.then()` caller. This seems to be fairly common in existing implementations of similar functionality. This composes reasonably well with non-standard executor implementations, in that there is little inherent overhead in running the continuation and the continuation code then has the option to explicitly run the bulk of the computation with a suitable home-brew executor. It's also the right approach when the continuation is known to complete almost immediately.
2. Since the continuation may run a long time, and it would be surprising to block the `.then()`-caller, we could instead run the continuation in a separate thread, at least in this case. This was the preference expressed by SG1 during the LWG2533 discussion. It almost certainly produces the least surprising results in the absence of home-brew executors.

As far as I have been able to determine, option (1) is more commonly implemented, but a quick inspection of some client code suggests that it's common to overlook the future-already-ready case, and erroneously assume that the continuation will run in the task satisfying the promise. If the user does carefully consider this case, it is

often difficult to handle well without a facility to cheaply move the continuation to an execution agent running concurrently with the `.then()`. Executors are intended to provide that facility, but are not yet ready.

I'm currently suspicious that option (1), when used by typical programmers, tends to introduce unexpected thread delays due to long-latency operations running on the wrong thread.

SG1 favored option (2) for similar reasons. But that is unlikely to remain optimal once executor use is widespread. For users who want to control where the continuation is run, it may require the creation of a thread-like execution agent, solely to invoke an executor, which then will typically use an executor to run the bulk of the continuation where it should really run. Current users with an existing executor implementation seem to dislike this option, for good reason.

Thus we have two possible interpretations of the `.then()` semantics, neither of which is clearly the right default behavior. And the correct default here, if there is one, seems intimately tied to the design of executors.

Possible routes forward

For the sake of discussion, here are some possible options for the committee:

1. Delay until the executor design is reasonably complete.
2. Proceed to move this clause of the Concurrency TS into the C++20 working draft, while agreeing on e.g. SG1's resolution of LWG2533.
3. Proceed to move this clause into the C++20 working draft with functionality similar to the current `.then()`, but with a change to the calling syntax that does not position it as the default `.then()` operation once we have executors. We could either rename the function, or add a required parameter that better specifies its behavior.

I very tentatively favor the last option here, which could allow us to support both option 1 and option 2 from the previous section. Syntax like `f.then(std::maybe_now, continuation)` would defang option 1 from the last section.